

# AL-AZHARI LIBRARY

مكتبة الأزهرى

## MongoDB Database Implementation Plan

Production database architecture, data model, collections, indexes, validation, state management, and operational strategy.

|           |                                                                                              |
|-----------|----------------------------------------------------------------------------------------------|
| VERSION   | 1.0                                                                                          |
| STATUS    | Implementation Blueprint / Draft for Engineering Review                                      |
| PREPARED  | 22 September 2026                                                                            |
| STACK     | MongoDB Atlas · Node.js · Express · TypeScript · Mongoose · Angular · Cloudinary · Socket.IO |
| AUTHORITY | PRD v1.0, Master Product Brief v2.0, IA/Sitemap, UX/UI Specification, Design System          |

# Contents

---

## Contents

- 1. Document Control
  - 1.1 Source labels
- 2. Database Architecture Overview
  - 2.1 Principles
  - 2.2 Deployment assumptions
- 3. Domain Model
- 4. Collection Inventory
  - 4.1 Candidate collections deliberately not created
- 5. Collection-by-Collection Specifications
  - 5.1 users
  - 5.2 addresses
  - 5.3 roles
  - 5.4 categories
  - 5.5 products
  - 5.6 carts
  - 5.7 orders
  - 5.8 payments
  - 5.9 paymentProofs
  - 5.10 inventoryReservations
  - 5.11 inventoryTransactions
  - 5.12 serviceCategories
  - 5.13 serviceRequests
  - 5.14 quotations
  - 5.15 preOrders
  - 5.16 returnRequests
  - 5.17 refunds
  - 5.18 notifications
  - 5.19 coupons
  - 5.20 couponRedemptions
  - 5.21 contentModules
  - 5.22 settings
  - 5.23 auditLogs

- 5.24 outboxEvents
- 6. Schema Design
  - 6.1 Common conventions
  - 6.2 Money
  - 6.3 Dates
- 7. Embedded vs Referenced Data Decisions
- 8. Relationships and Referential Integrity
- 9. State Enums
- 10. Index Strategy
  - 10.1 Search
  - 10.2 Tradeoffs
- 11. Unique Constraints
- 12. Validation Rules
- 13. Inventory / Reservation Data Model
- 14. Order Data Model
- 15. Payment / Proof Data Model
- 16. Service / Quotation Data Model
- 17. Return / Refund Data Model
- 18. Notification Data Model
- 19. RBAC Data Model
- 20. Audit Log Data Model
- 21. Coupon Data Model
- 22. Content / Merchandising Data Model
- 23. Reporting Considerations
- 24. Transactions / Atomic Operations
- 25. Concurrency Strategy
- 26. Idempotency Strategy
- 27. Data Lifecycle / Retention
- 28. Security and Privacy
- 29. MongoDB Atlas Considerations
- 30. Seed Data Strategy
- 31. Migration Strategy
- 32. Example Documents
  - 32.1 User
  - 32.2 Product with variant
  - 32.3 Cart
  - 32.4 Order

- 32.5 Payment and proof
  - 32.6 Service request and quotation
  - 32.7 Pre-order, return, refund, notification, audit
- 33. Example Queries
- 34. Index Definitions
- 35. Database Testing Strategy
  - Unit and schema tests
  - Integration tests
  - Concurrency tests
  - Data security tests
- 36. Traceability Matrix
- 37. Open Decisions
- 38. Implementation Checklist
- DOCUMENT INTEGRITY REPORT
  - Requirements reviewed
  - Confirmed implementation requirements
  - Technical recommendations introduced
  - Open Decisions preserved
  - Potential schema/backend gaps
  - Potential contradictions
  - Security risks
  - Implementation ambiguities
  - Final consistency status

**Reading the document.** CONFIRMED REQUIREMENT items are preserved from the supplied product sources. TECHNICAL RECOMMENDATIONS describe implementation mechanisms. OPEN DECISIONS remain configurable and must be finalized before production behavior is hard-coded.

# 1. Document Control

| Field                | Value                                                                    |
|----------------------|--------------------------------------------------------------------------|
| Product              | AL-AZHARI LIBRARY — Online Store & Student Services Platform             |
| Version              | 1.0                                                                      |
| Status               | Architecture and implementation handoff; <b>PASS WITH OPEN DECISIONS</b> |
| Prepared             | 22 September 2026                                                        |
| Database             | MongoDB Atlas                                                            |
| Currency             | EGP; integer minor units ( <code>amountMinor</code> )                    |
| Time persistence     | UTC BSON <code>Date</code> ; localize only at the client                 |
| Functional authority | PRD v1.0                                                                 |
| Business authority   | Master Product Brief v2.0                                                |
| UX/route authority   | UX/UI Specification v1.0 and IA/Sitemap                                  |
| Related plan         | <code>BACKEND-IMPLEMENTATION-PLAN.md</code>                              |

This document describes data architecture only. It does not create an automated payment gateway, service-file upload system, reviews, ratings, LMS, marketplace, phone OTP, map pins, or a second service checkout.

## 1.1 Source labels

- **CONFIRMED REQUIREMENT** — required by the supplied PRD, brief, IA, or UX specification.
- **TECHNICAL RECOMMENDATION** — implementation guidance that does not create a new business rule.
- **OPEN DECISION** — unresolved source behavior that must remain configurable or explicitly pending.

# 2. Database Architecture Overview

MongoDB is the persisted source of truth. Socket.IO, email, WhatsApp, Cloudinary, and frontend state are delivery or integration layers only.

## 2.1 Principles

1. Embed bounded data that is read and changed with its owner.
2. Reference independently managed, high-volume, sensitive, or immutable records.
3. Snapshot mutable catalog/customer/fulfillment facts in financial and operational records.

4. Store money as integer EGP minor units; never use floating point.
5. Store dates as UTC BSON `Date`.
6. Enforce state transitions in services and transactions, not through arbitrary status updates.
7. Keep payment proofs as Cloudinary references and metadata, never binary data.
8. Keep service requests out of carts and product orders.
9. Keep audit logs, notifications, inventory movements, and proof history out of unbounded business-document arrays.
10. Prefer one MongoDB deployment with modular collections over microservices.

## 2.2 Deployment assumptions

- MongoDB Atlas replica set, because multi-document transactions are required for acceptance, fulfillment, and refund operations.
- Application uses the official MongoDB Node.js driver through a small repository/data-access layer. Mongoose is acceptable only if its schemas preserve the same constraints and transaction behavior.
- Atlas Search is not required for the initial implementation. The initial search uses normalized fields, equality/range queries, and a controlled text strategy; see §10.

## 3. Domain Model

```
User —< Address
User —< Cart —< CartItem >— Product —< embedded Variant
User/Guest —< Order —< OrderItem >— Product/Variant snapshots
Order —1 Payment —< PaymentProof
Order —< InventoryReservation —< InventoryTransaction
Order —< ReturnRequest —0..1 Refund
User —< ServiceRequest >— ServiceCategory
ServiceRequest —< Quotation —0..1 Payment —< PaymentProof
User —< PreOrder >— Product/Variant
User —< Notification
Admin action —< AuditLog
Coupon —< CouponRedemption >— Order
ContentModule —> Product/Category
```

`User` is optional on guest orders and required on registered account records. Orders and service requests retain contact snapshots so historical records remain reconstructable after profile edits.

## 4. Collection Inventory

| Collection              | Required | Ownership     | Why it exists                                                       |
|-------------------------|----------|---------------|---------------------------------------------------------------------|
| <code>users</code>      | Yes      | Auth/customer | Account identity, role assignment, verification state               |
| <code>addresses</code>  | Yes      | Customer      | Reusable saved addresses; checkout snapshots are embedded in orders |
| <code>roles</code>      | Yes      | RBAC          | Extensible role-to-permission assignment                            |
| <code>categories</code> | Yes      | Catalog       | Stable, books-first taxonomy                                        |

| Collection            | Required    | Ownership             | Why it exists                                                              |
|-----------------------|-------------|-----------------------|----------------------------------------------------------------------------|
| products              | Yes         | Catalog/inventory     | Product identity, content, embedded variants, publication                  |
| carts                 | Yes         | Commerce              | Guest/session and registered persisted carts                               |
| orders                | Yes         | Commerce              | Authoritative product transaction and lifecycle                            |
| payments              | Yes         | Payments              | Financial payment state for an order or accepted quotation                 |
| paymentProofs         | Yes         | Payments              | Separate proof submissions and review history                              |
| inventoryReservations | Yes         | Inventory             | Reservation ownership and release/finalization                             |
| inventoryTransactions | Yes         | Inventory             | Immutable stock movement ledger                                            |
| serviceCategories     | Yes         | Services              | Configurable service catalog and form definitions                          |
| serviceRequests       | Yes         | Services              | Authoritative service request lifecycle                                    |
| quotations            | Yes         | Services              | Versionable final quote and customer decision                              |
| preOrders             | Yes         | Commerce              | Unavailable-product demand separate from stock                             |
| returnRequests        | Yes         | Returns               | Item-level return workflow                                                 |
| refunds               | Yes         | Returns/payments      | Manual refund record distinct from payment/proof                           |
| notifications         | Yes         | Cross-cutting         | Persisted lifecycle notifications                                          |
| coupons               | Yes         | Discounts             | Coupon definitions and current usage controls                              |
| couponRedemptions     | Recommended | Discounts/reporting   | Unique, auditable order/coupon usage records                               |
| contentModules        | Yes         | Content               | Homepage and seasonal merchandising without taxonomy mutation              |
| settings              | Recommended | Configuration         | Payment methods, contact, shipping, policy flags, and other open decisions |
| auditLogs             | Yes         | Compliance/operations | Immutable critical-action history                                          |
| outboxEvents          | Recommended | Integration delivery  | Reliable post-commit email/realtime work without Redis                     |

## 4.1 Candidate collections deliberately not created

- productVariants** : variants are bounded and always presented with a product; embed them in **products** . Orders and inventory retain variant snapshots/IDs.
- shipping** : shipping estimate, final cost, provider, and status belong to the order fulfillment snapshot. Shipping rate/configuration is held in **settings** until OD-03-06 are resolved.
- permissions** : permission keys are stable configuration values embedded in **roles** ; a separate collection adds no value at the initial scale.
- serviceAttachments** : explicitly prohibited. Files are exchanged through WhatsApp/Telegram.
- reviews** , **ratings** , **gatewayTransactions** : out of scope.

## 5. Collection-by-Collection Specifications

The field tables below define the minimum contract. Unless stated otherwise, every collection has `createdAt` and `updatedAt` as UTC dates.

### 5.1 users

**Purpose:** registered Customer, Admin, and Owner identities. **Source:** AUTH-002-012, RBAC-001-004. **Referenced by:** addresses, carts, orders, service requests, notifications, audit logs.

| Field                            | Type      | Req. | Example / validation                                                                        |
|----------------------------------|-----------|------|---------------------------------------------------------------------------------------------|
| <code>_id</code>                 | ObjectId  | Yes  | Generated                                                                                   |
| <code>role</code>                | string    | Yes  | <code>customer</code> , <code>admin</code> , <code>owner</code> ; configurable future roles |
| <code>name</code>                | string    | Yes  | Trimmed, non-empty                                                                          |
| <code>email</code>               | string    | Yes  | Lowercase, unique, valid format                                                             |
| <code>phone</code>               | string    | Yes  | Canonicalized Egyptian/international format; unique                                         |
| <code>passwordHash</code>        | string    | Yes  | Argon2id/bcrypt output; never plaintext                                                     |
| <code>emailVerifiedAt</code>     | Date/null | Yes  | Null until verified                                                                         |
| <code>status</code>              | string    | Yes  | <code>active</code> , <code>suspended</code>                                                |
| <code>lastLoginAt</code>         | Date/null | No   | UTC                                                                                         |
| <code>refreshTokenVersion</code> | int       | Yes  | Increment on global logout/revocation                                                       |

Sensitive fields are never returned by default. Email verification/reset tokens are stored hashed in separate short-lived token records or a bounded `authTokens` subdocument; never store raw tokens.

### 5.2 addresses

**Purpose:** saved customer addresses. **Source:** AUTH-011, CHK-002-006. **Security:** owner-only access.

Fields: `userId` , `label` , `recipientName` , `recipientPhone` , `governorate` , `city` , `area` , `street` , `buildingNumber` , `floor` , `apartment` , `landmark` , `notes` , `isDefault` . All address fields are strings except `isDefault` . No map coordinates or location-pin fields.

### 5.3 roles

**Purpose:** permission-based authorization. **Source:** RBAC-001-004. Fields: `key` , `displayName` , `permissionKeys[]` , `isSystem` , `active` . `permissionKeys` are bounded strings such as `orders.read` ; candidate keys remain configurable recommendations, not owner-approved business policy.

### 5.4 categories

**Purpose:** stable catalog taxonomy. **Source:** CAT-004-009, CMS-002. Fields: `slug` (unique), localized `name` , `parentId` (nullable), `kind` ( `product` ), `displayOrder` , `isActive` , `isMvpEnabled` , `isBooksCore` . Services use `serviceCategories` , not this collection. Future categories may exist inactive but must not be purchasable in MVP.

## 5.5 products

**Purpose:** published catalog products and embedded variant/inventory records. **Source:** CAT-001–009, VAR-001–003, SEARCH-001–004, PDP-001–005, INV-001.

Top-level fields: `slug`, localized `name`, `description` nullable, `categoryId`, `images[]`, searchable metadata (`author`, `grade`, `stage`, `subject`, `publisher`, `isbn`, `educationType`), `hasVariants`, `availability`, `priceMinor`, `currency`, `preOrderEligible`, `isPublished`, `returnPolicyFlags`, `searchText`, `createdAt`, `updatedAt`.

Embedded `variants[]` fields: `variantId` (UUID/string stable within product), `attributes` object, localized `label`, `priceMinor`, `currency`, `availability`, `stockTotal`, `stockReserved`, `preOrderEligible`, `sku` optional, `images[]` optional, `updatedAt`.

Validation:

- Product has either a top-level price or at least one usable variant price.
- `priceMinor`, stock counters, and quantities are non-negative integers.
- `stockAvailable` is derived as `stockTotal - stockReserved`; do not persist a mutable duplicate unless guarded by the same transaction.
- `availability` is `in_stock`, `out_of_stock`, or `pre_order_eligible`.
- Descriptions and optional metadata may be null.
- No review/rating fields.
- Variant attributes are product-specific; do not require all attribute types.

## 5.6 carts

**Purpose:** fixed-price product cart. Fields: `ownerType` (`guest`, `user`), `userId` nullable, `sessionId` nullable, `items[]`, `currency`, `expiresAt`, `version`, timestamps.

Each item contains `productId`, `variantId` nullable, `quantity`, `unitPriceMinor` (display snapshot only), `productNameSnapshot`, `imageSnapshot`, and `addedAt`. On checkout, authoritative current product data is read again if CART-005 is approved/implemented. Services are rejected at the API boundary.

## 5.7 orders

**Purpose:** authoritative product order. **Source:** ORD-001–009, CHK-001–012, SHIP-001–009, PICK-001–004.

Required fields:

```

reference
customerId|null
guestAccessTokenHash|null
customerSnapshot { name, email|null, phone }
items[]
totals { productSubtotalMinor, shippingEstimateMinor, shippingFinalMinor|null,
         discountMinor, totalMinor, currency }
fulfillment {
  method: "delivery"|"pickup",
  addressSnapshot|null,
  provider|null,
  shippingStatus,
  estimateSource|null,
  finalCostConfirmedAt|null
}
paymentMethodKey
paymentId|null
status
statusHistory[] (bounded lifecycle history; audit remains external)
couponSnapshot|null
submittedAt
acceptedAt|null
completedAt|null
cancelledAt|null
idempotencyKey
version

```

An order item contains immutable `productId`, `variantId`, name/category/image snapshots, selected attributes, `quantity`, `unitPriceMinor`, `lineTotalMinor`, `availabilityAtSubmission`, and `stockItemKey`. The order stores all amounts used to reconstruct the financial record. `shippingEstimateMinor` and `shippingFinalMinor` are distinct; `shippingFinalMinor` is null until Admin records it.

status values: `pending_review`, `accepted`, `awaiting_payment`, `payment_verification`, `awaiting_new_proof`, `payment_confirmed`, `customer_confirmation_required`, `confirmed`, `preparing`, `ready_for_pickup`, `picked_up`, `shipped`, `out_for_delivery`, `delivered`, `completed`, `rejected`, `cancelled`, `returned`.

## 5.8 payments

**Purpose:** financial state for an order or accepted service quotation. **Source:** PAY-001-008, QUOTE-003/005, REF-001-003.

Fields: `ownerType` (`order`, `serviceQuotation`), `ownerId`, `customerId|null`, `methodKey`, `methodSnapshot`, `amountDueMinor`, `currency`, `status`, `proofRequired`, `proofSubmissionCount`, `confirmedAt|null`, `rejectedAt|null`, `metadata` excluding secrets and transaction IDs, timestamps.

Payment statuses: `not_submitted`, `proof_uploaded`, `under_review`, `confirmed`, `rejected`, `new_proof_requested`. Refund states belong to `refunds`, not this document.

## 5.9 paymentProofs

**Purpose:** one customer proof submission containing one or more screenshot references. **Source:** PAY-002, PAY-004-008.

Fields: `paymentId`, `ownerType`, `ownerId`, `customerId|null`, `submissionNumber`, `files[]`, `status` (`uploaded`, `under_review`, `confirmed`, `rejected`, `new_proof_requested`), `customerNote|null`, `reviewNote|null`, `reviewedBy|null`, `reviewedAt|null`, `createdAt`.

Each file contains `cloudinaryPublicId`, `resourceType`, `format`, `bytes`, `width`, `height`, `sha256` if available, and secure delivery metadata. Do not store public unsigned URLs. Authorized Admin/Owner retrieval creates a short-lived signed URL on demand.

## 5.10 inventoryReservations

**Purpose:** order acceptance reservations. **Source:** INV-002-006, OD-08, OD-21.

Fields: `orderId`, `orderItemId`, `productId`, `variantId|null`, `quantity`, `status` (`active`, `released`, `consumed`), `createdAt`, `releasedAt|null`, `consumedAt|null`, `releasedReason|null`. One active reservation per order item is enforced by a partial unique index.

## 5.11 inventoryTransactions

**Purpose:** immutable stock ledger and reporting. Fields: `productId`, `variantId|null`, `type` (`manual_adjustment`, `reservation`, `release`, `deduction`, `correction`), `quantityDelta`, `before`, `after`, `sourceType`, `sourceId`, `actorId|null`, `reason`, `createdAt`. Never update ledger rows; append corrections.

## 5.12 serviceCategories

**Purpose:** service catalog and configurable fields. **Source:** SRV-001-008, OD-12-14.

Fields: `slug`, localized `name`, localized `description`, `kind` (`printing`, `photocopying`, `binding`, `applications_transfers`, `research_formatting`, `other_admin`), `isActive`, `formVersion`, `fields[]`, `communicationChannels[]`, `pricingMode` (`quotation`), `turnaroundText|null`, `codAllowed|null`, timestamps.

`fields[]` stores only approved service form definitions: `key`, localized label, `type`, `required`, `options[]`, `active`. Until OD-12 is resolved, field definitions remain empty/configurable; do not invent customer fields. `turnaroundText` and `codAllowed` remain null until approved.

## 5.13 serviceRequests

**Purpose:** authoritative service lifecycle. **Source:** SRV-001-008, UX SRV-003-006.

Fields: `reference`, `customerId|null`, `customerSnapshot`, `serviceCategoryId`, `serviceCategorySnapshot`, `submittedFields` object, `description`, `status`, `quotationId|null`, `paymentId|null`, `communicationContext`, `statusHistory[]`, `closedReason|null`, timestamps.

Statuses: `submitted`, `admin_review`, `quotation_sent`, `awaiting_payment`, `payment_verification`, `payment_confirmed`, `processing`, `completed`, `closed_not_proceeding`, `closed_declined`. `submittedFields` is bounded by the active service configuration. No file or attachment field is allowed.

## 5.14 quotations

**Purpose:** final quote and customer decision. **Source:** QUOTE-001-005, OD-14.

Fields: `serviceRequestId`, `customerId`, `version`, `amountMinor`, `currency`, `status` (`draft`, `sent`, `accepted`, `rejected`, `expired` only if a policy is later approved), `customerDecisionAt|null`, `decisionNote|null`, `sentBy`, `acceptedAt|null`, `rejectedAt|null`, `paymentId|null`, `createdAt`, `updatedAt`.

Only `sent` quotations are customer-visible. Payment creation is gated by `accepted`. Auto-closing a rejected quote is a technical/product recommendation from the PRD and must remain configurable until approved.

## 5.15 preOrders

**Purpose:** demand for eligible unavailable products. **Source:** PRE-001-007, OD-10-11.

Fields: `reference`, `customerId|null`, `customerSnapshot`, `productId`, `variantId|null`, `productSnapshot`, `quantity`, `capturedPriceMinor`, `currency`, `status`, `expectedAvailabilityAt|null`, `paymentId|null`,

linkedOrderId|null, allocationSequence|null, timestamps.

Statuses: requested, admin\_review, accepted, payment\_pending, payment\_verification, confirmed, available, rejected, fulfilled, cancelled. Do not assign allocationSequence until OD-11 is approved. Pre-order acceptance creates no inventory reservation.

## 5.16 returnRequests

**Purpose:** item-level return workflow. **Source:** RET-001-004.

Fields: reference, orderId, customerId, items[] (orderId, quantity, reason, eligible, evidenceMetadata[]), status, customerNote, adminNote, reviewedBy|null, reviewedAt|null, refundId|null, timestamps.

Statuses: return\_requested, return\_review, return\_approved, refund\_initiated, refund\_completed, return\_rejected. Evidence rules and retention remain configurable; service files are not uploaded here.

## 5.17 refunds

**Purpose:** manual refund record. **Source:** REF-001-003, OD-16.

Fields: orderId, returnRequestId, customerId, amountMinor, currency, methodKey, status (initiated, completed, failed), recordedBy, recordedAt, completedAt|null, note, attemptReference|null (not a customer transaction-ID field), timestamps. No automated gateway behavior.

## 5.18 notifications

**Purpose:** persisted lifecycle notification. **Source:** NOT-001-004, RT-001-003.

Fields: recipientUserId, recipientRoleContext, type, localized title/body, entityType, entityId, actionUrl, readAt|null, channels (in\_app, email, socket), deliveryStatus, dedupeKey, timestamps. Do not embed indefinitely in users/orders.

## 5.19 coupons

**Purpose:** percentage/fixed, seasonal, product/category-scoped discounts. **Source:** COUP-001-005, OD-17.

Fields: codeNormalized, discountType (percentage, fixed), value, currency|null, scopeType (order, product, category), scopeIds[], active, startsAt|null, endsAt|null, usageCount, usageLimit|null, minimumOrderMinor|null, stackable|null, customerRestriction nullable, version, timestamps.

Fields controlled by OD-17 remain null/configurable. value is basis points for percentage (e.g. 1500 = 15%) or minor units for fixed discount.

## 5.20 couponRedemptions

**Purpose:** one immutable redemption per order/coupon; supports reports and idempotency. Fields: couponId, codeSnapshot, orderId, customerId|null, discountMinor, createdAt. Unique (couponId, orderId).

## 5.21 contentModules

**Purpose:** homepage, seasonal banners, featured products/categories. **Source:** CMS-001-004. Fields: key, localized title/body, moduleType, productIds[], categoryIds[], startsAt, endsAt, displayOrder, active, updatedBy. Never mutate the permanent taxonomy through content.

## 5.22 settings

**Purpose:** data-driven operational configuration. **Source:** OD-01–20, CHK-007, SHIP-003/007. Fields: `key`, `value` (validated by key-specific schema), `version`, `updatedBy`, timestamps.

Initial keys include `payment.methods`, `payment.details`, `contact.whatsapp`, `contact.phone`, `shipping.rateProvider`, `shipping.serviceability`, `shipping.policy`, `return.policy`, `coupon.policy`, `service.policy`, and `auth.verificationPolicy`. Secrets do not belong here; use environment secrets.

## 5.23 auditLogs

**Purpose:** immutable critical-operation record. **Source:** PAY-007, ADM-002/003, RBAC, operational workflows.

Fields: `actorId|null`, `actorRole`, `action`, `entityType`, `entityId`, `previousState|null`, `newState|null`, `metadata`, `requestId`, `ipHash|null`, `createdAt`. Never expose raw payment proof URLs, passwords, tokens, or unnecessary PII.

## 5.24 outboxEvents

**Purpose:** post-commit delivery work. **TECHNICAL RECOMMENDATION.** Fields: `eventType`, `aggregateType`, `aggregateId`, `payload`, `dedupeKey`, `status` (`pending`, `processing`, `sent`, `failed`), `attempts`, `availableAt`, `processedAt|null`, `lastError|null`, timestamps. This prevents email/socket delivery failure from rolling back the business operation.

# 6. Schema Design

---

## 6.1 Common conventions

- `ObjectId` for internal references.
- Stable public `reference` strings for orders, services, pre-orders, returns, and refunds.
- Slugs are lowercase, URL-safe, and unique among active records.
- Localized content uses `{ ar: string, en?: string }`; English may be absent at launch.
- Nullable business fields are explicitly `null`, not ambiguous empty strings.
- All persisted user input is trimmed and validated before storage.

## 6.2 Money

All money fields are integer minor units in EGP ( `amountMinor: 12500` means EGP 125.00 if the configured EGP minor-unit convention is piastres). The frontend formats values with locale-aware EGP formatting. A `currency: "EGP"` snapshot is stored wherever money is recorded. No floating-point amounts are accepted by API DTOs.

## 6.3 Dates

MongoDB BSON dates are UTC. Store `createdAt`, `updatedAt`, `stateChangedAt`, and domain timestamps such as `submittedAt`, `acceptedAt`, `confirmedAt`, `completedAt`, `reviewedAt`, and `recordedAt`. The client formats Africa/Cairo dates.

# 7. Embedded vs Referenced Data Decisions

| Data                     | Decision                                          | Reason                                                                         |
|--------------------------|---------------------------------------------------|--------------------------------------------------------------------------------|
| Product variants         | Embed in products                                 | Bounded, product-owned, read together; avoids variant collection joins         |
| Order items              | Embed snapshots in orders                         | Historical financial truth must survive catalog changes                        |
| Order address/contact    | Embed snapshot                                    | Customer edits must not rewrite historical fulfillment                         |
| Order status history     | Embed bounded lifecycle entries; audit externally | Small, customer-readable timeline; detailed audit remains unbounded collection |
| Payment proofs           | Reference                                         | Sensitive media, re-upload history, potentially multiple files                 |
| Inventory ledger         | Reference                                         | Append-only and high-volume                                                    |
| Notifications            | Reference                                         | Unbounded per user                                                             |
| Service submitted fields | Embed                                             | Request-specific bounded data                                                  |
| Service form definition  | Embed in category version                         | Rendered request must retain the field-version context                         |
| Audit logs               | Reference                                         | Immutable and unbounded                                                        |
| Content links            | Reference IDs                                     | Products/categories are independently managed                                  |

# 8. Relationships and Referential Integrity

MongoDB does not enforce foreign keys. Repositories must validate referenced IDs and state ownership before writes. Deactivation is preferred over deletion for products, categories, services, coupons, and users with historical records. Business records are not hard-deleted through normal Admin UI.

# 9. State Enums

The application must expose typed constants matching §5. Order, service, payment, pre-order, return, and refund transitions are authoritative in the backend plan. Unknown status strings must be rejected at write time.

# 10. Index Strategy

## 10.1 Search

**MVP confirmed requirement:** use MongoDB indexing/query capabilities; do not introduce Elasticsearch or another dedicated engine.

Recommended initial strategy:

- equality indexes for `categoryId`, `availability`, `educationType`, `stage`, `grade`, `subject`, `publisher`;
- range index for `priceMinor`;
- normalized exact/prefix fields for ISBN and slug;
- a controlled `searchText` field containing populated searchable values, queried with a text index only if Arabic analyzer behavior is validated;
- for larger catalog scale, **TECHNICAL RECOMMENDATION:** Atlas Search with an Arabic-capable analyzer and explicit field weights, but do not claim ranking behavior until approved/tested.

Missing optional metadata remains absent/null and must not make other populated fields undiscoverable. Do not create an index for every optional field without query evidence.

## 10.2 Tradeoffs

- Compound indexes should match actual filter/sort combinations; too many indexes slow Admin product edits.
- Text search on Arabic may require Atlas Search validation; fallback is field-specific regex/prefix search with bounded result limits.
- Partial indexes reduce index size for active/public records.

## 11. Unique Constraints

---

- `users.email` unique, case-normalized.
- `users.phone` unique, canonicalized.
- `categories.slug` unique.
- `products.slug` unique.
- `products.isbn` unique only when non-null and policy confirms ISBN uniqueness.
- `orders.reference` unique.
- `orders.idempotencyKey` unique per authenticated owner/session scope.
- `carts.userId` unique for active user cart; guest carts use session uniqueness.
- `preOrders.reference`, `serviceRequests.reference`, `returnRequests.reference` unique.
- `coupons.codeNormalized` unique.
- `couponRedemptions(couponId, orderId)` unique.
- `paymentProofs(paymentId, submissionNumber)` unique.
- active `inventoryReservations(orderId, orderItemId, status=active)` unique via partial index.

## 12. Validation Rules

---

At the database boundary and application boundary:

- reject negative prices, totals, quantities, stock, discount values, and refund amounts;
- require `quantity >= 1`;
- require a valid `ObjectId` for internal references;
- require product/variant availability before cart/checkout;
- require payment proof for configured non-COD methods and reject proof for COD;

- reject service file fields;
- require `amountMinor` and `currency` together;
- require order item snapshots and totals before order creation;
- do not permit customer writes to authoritative status fields;
- reject state transitions not in the server transition map;
- require actor and reason for critical Admin decisions where the product captures a reason;
- enforce role ownership on all customer-scoped queries.

## 13. Inventory / Reservation Data Model

For a normal product/variant:

```
available = stockTotal - stockReserved
```

The acceptance transaction conditionally increments `stockReserved` only when enough availability remains. It creates an `inventoryReservations` row and ledger row in the same transaction. Rejection/cancellation decrements reserved stock and marks the reservation released. Delivered/Picked Up decrements total and reserved stock and marks the reservation consumed.

**OPEN DECISION:** concurrency fairness (OD-08), partial acceptance (OD-21), and thresholds (OD-09). The safe baseline is atomic conditional reservation and no negative availability; it must not silently implement partial acceptance.

## 14. Order Data Model

Order creation validates product references, selected variants, prices, availability, fulfillment, payment method configuration, coupon result, and guest/customer contact. It stores snapshots and enters `pending_review`. It does not reserve stock.

Editing an order is implemented as a controlled replacement of editable items/details while status is `pending_review`; it recalculates totals and increments `version`. After acceptance, no financial fields are customer-editable. A unique idempotency key prevents duplicate order creation.

## 15. Payment / Proof Data Model

Payment method definitions are read from `settings` and snapshotted into `payments` and orders. The payment-method list remains **OPEN DECISION OD-20**: the PRD lists InstaPay, Vodafone Cash, Orange Cash, Etisalat Cash, WE Pay, and COD; the Master Brief confirms COD, cash at library, InstaPay, and Vodafone Cash.

Proof workflow:

1. Customer creates a proof submission after the order is accepted and payment is awaiting.
2. Backend authorizes the payment/order owner.
3. Controlled Cloudinary upload stores private references.
4. Backend records `paymentProofs` and changes payment/order state in a transaction.

5. Admin confirms, rejects, or requests new proof.

6. Every review action creates an audit log and notification/outbox event.

No transaction-ID field is persisted as customer input.

## 16. Service / Quotation Data Model

---

Service requests and quotations are separate from product orders. Service forms are configuration-driven and versioned. The request stores the submitted field values plus the category/form snapshot. No service attachment is accepted by schema, multipart route, or Cloudinary folder.

Quotation acceptance is a transaction: conditionally change quotation from `sent` to `accepted`, change request to `awaiting_payment`, and create or activate a payment record. Rejection does not create payment. Whether accepted services allow COD is **OPEN DECISION OD-14**.

## 17. Return / Refund Data Model

---

A return can be created only for a completed order and eligible item. Admin policy flags determine whether an item is returnable; periods and detailed rules are **OPEN DECISION OD-15**. Approval and refund initiation are distinct states. Recording a manual refund creates a separate `refunds` document and updates the return in one transaction. Refund processing SLA is **OPEN DECISION OD-16**.

## 18. Notification Data Model

---

Notifications are created from committed state changes. Use deterministic `dedupeKey` values such as `order:{id}:status:{status}:{version}`. Socket.IO and email consume the persisted notification/outbox event. A failed delivery never rolls back the order/payment/service transaction.

## 19. RBAC Data Model

---

Initial roles: `owner`, `admin`, `customer`. Store permission keys on roles to support future specialized roles without redesign. Permission candidates are configuration, not an invented Owner-only policy. Authorization checks both permission and record ownership.

## 20. Audit Log Data Model

---

Audit critical mutations only: order accept/reject/status changes, payment proof review, inventory adjustments, shipping changes, quotation actions, return decisions, refunds, role/permission changes, and settings changes. Audit logs are append-only; no user-facing endpoint can edit or delete them.

## 21. Coupon Data Model

Coupon application is calculated against a server-side order snapshot. The result is stored in the order and a redemption row is created only after successful order creation. Stacking, limits, expiry defaults, minimums, and customer restrictions remain configurable under OD-17. Never infer them from a coupon code.

## 22. Content / Merchandising Data Model

`contentModules` supports featured books/categories, seasonal banners, and descriptions without changing permanent taxonomy. Date-windowed modules are optional presentation layers. The data model must allow the year-round catalog to remain discoverable when no module is active.

## 23. Reporting Considerations

Reports read from orders, order snapshots, payments, services, pre-orders, coupon redemptions, and inventory ledger. Use aggregation pipelines with date and status filters. Required dimensions: orders, revenue, outside-Qena geography, payment methods, service request/quotation conversion, product/category demand, pre-order demand, and coupon usage. No target gauges or invented KPI thresholds; OD-18 remains open.

For high-volume reporting, **TECHNICAL RECOMMENDATION:** build read-only daily aggregates later rather than slowing transactional queries. Do not add them before query evidence exists.

## 24. Transactions / Atomic Operations

Use MongoDB transactions only where multiple authoritative documents must change together:

| Operation                                            | Transaction | Reason                                        |
|------------------------------------------------------|-------------|-----------------------------------------------|
| Create order + initial payment + notification/outbox | Yes         | Prevent partial financial/order creation      |
| Accept order + reserve every line                    | Yes         | Prevent overselling and orphaned reservations |
| Reject/cancel + release reservation                  | Yes         | Keep stock and state aligned                  |
| Delivered/Picked Up + final stock deduction          | Yes         | Prevent duplicate deduction                   |
| Submit payment proof + payment/order state           | Yes         | Keep proof status and order state aligned     |
| Admin payment review                                 | Yes         | State, audit, notification must agree         |
| Accept quotation + payment availability              | Yes         | No payment before acceptance                  |
| Return approve + refund initiation                   | Yes         | Keep return/refund linkage aligned            |
| Record refund completion                             | Yes         | Refund record and return state must agree     |
| Product content edit alone                           | No          | Single-document update                        |
| Read/search/report                                   | No          | No cross-document write                       |

| Operation              | Transaction | Reason                 |
|------------------------|-------------|------------------------|
| Mark notification read | No          | Single-document update |

Transactions should be short, use conditional updates, and emit outbox events in the same transaction where delivery must reflect the committed change.

## 25. Concurrency Strategy

- **Last stock:** conditional atomic update on the product/variant stock counters; transaction aborts on insufficient availability.
- **Simultaneous Admin acceptance:** update order only when current status is `pending_review` and reserve with a version/availability condition. A second acceptance receives a conflict.
- **Duplicate checkout:** require idempotency key; unique index and replay the original result.
- **Repeated proof upload:** unique payment/submission number plus idempotency key; do not create duplicate active reviews.
- **Duplicate notification:** deterministic dedupe key and unique index.
- **Inventory adjustment:** transaction plus version predicate; reject stale Admin writes.
- **Concurrent order edits:** optimistic `version` check; customer must reload on conflict.
- **Pre-order allocation:** do not assign fairness logic until OD-11 is resolved.

## 26. Idempotency Strategy

State-changing endpoints accept `Idempotency-Key` where repeated submission can create a financial or operational record. Store a hash of method/path/body, owner scope, response status, response body reference, and expiry in the owning record or a dedicated key store if needed. At minimum cover checkout, proof submission, state transitions, refund completion, coupon redemption, and outbox event creation.

An idempotency replay returns the original result; a reused key with a different payload returns `IDEMPOTENCY_KEY_REUSED`.

## 27. Data Lifecycle / Retention

- Never hard-delete orders, payments, proofs, returns, refunds, or audit logs through normal UI.
- Soft-deactivate catalog/configuration records.
- Expire guest carts and raw auth tokens using TTL indexes.
- Keep payment proofs only for the approved operational/privacy retention period; this period is not defined by the sources and must be approved.
- Archive old notifications/outbox events only after delivery and approved retention.
- Cloudinary deletion must be coordinated with proof retention and audit history.

## 28. Security and Privacy

---

- Hash passwords with Argon2id or bcrypt; never log or store plaintext.
- Store refresh tokens hashed; rotate and revoke.
- Restrict payment proof reads to authorized Admin/Owner and the owning customer only where customer visibility is approved.
- Generate short-lived signed Cloudinary URLs server-side; never return API secrets or unrestricted public URLs.
- Use opaque order references and hashed guest access tokens.
- Enforce ownership filters to prevent IDOR.
- Minimize PII in logs and audit metadata; hash IPs if retained.
- Encrypt Atlas at rest and require TLS in transit.
- Keep secrets in environment/secrets management, never MongoDB documents or committed files.
- Use field projection to exclude password hashes, token hashes, proof storage internals, and private Admin notes.

## 29. MongoDB Atlas Considerations

---

- Use a replica set/tier supporting transactions.
- Restrict network access and database users by least privilege.
- Enable automated backups and point-in-time recovery according to the approved plan.
- Monitor query performance and index usage.
- Use Atlas metrics and alerts for connections, replication lag, storage, slow queries, and failed transactions.
- Test restore procedures before production.

## 30. Seed Data Strategy

---

Seed only:

1. initial `owner` / `admin` roles and permission configuration;
2. stable MVP categories: Books, School/Study Products, Other Products;
3. confirmed service categories;
4. payment methods as disabled/configurable until OD-20 is approved;
5. empty settings placeholders for contact, shipping, service, coupon, return, and verification policies.

Never seed fake products, prices, stock, phone numbers, payment details, delivery rates, or customer records.

## 31. Migration Strategy

---

1. Create collections and validators/indexes in a versioned migration runner.
2. Seed roles/categories/service definitions.
3. Import products with a dry-run report for missing price/image/category and duplicate slugs/ISBNs.

4. Import stock as an opening balance with `inventoryTransactions`.
5. Reconcile imported counts against the physical inventory before publishing.
6. Never migrate WhatsApp chats as authoritative orders without explicit data-entry review.
7. Make every migration idempotent and record a migration version.

## 32. Example Documents

### 32.1 User

```
{
  _id: ObjectId("66f0000000000000000000000001"),
  role: "customer",
  name: "أحمد علي",
  email: "ahmed@example.com",
  phone: "+201000000000",
  passwordHash: "$argon2id$v=19$...",
  emailVerifiedAt: null,
  status: "active",
  refreshTokenVersion: 0,
  createdAt: ISODate("2026-09-22T10:00:00Z"),
  updatedAt: ISODate("2026-09-22T10:00:00Z")
}
```

### 32.2 Product with variant

```
{
  _id: ObjectId("66f00000000000000000000000010"),
  slug: "azhar-arabic-reference-grade-3",
  name: { ar: "مرجع اللغة العربية - الصف الثالث", en: "Arabic Reference - Grade 3" },
  description: null,
  categoryId: ObjectId("66f00000000000000000000000020"),
  images: [{ urlKey: "catalog/azhar-arabic-3", alt: "مرجع اللغة العربية - الصف الثالث" }],
  author: null, grade: "3", stage: "secondary", subject: "arabic",
  publisher: "Example Publisher", isbn: null, educationType: "azhar",
  hasVariants: true, priceMinor: null, currency: "EGP",
  availability: "in_stock", preOrderEligible: false, isPublished: true,
  variants: [{
    variantId: "edition-2026",
    attributes: { edition: "2026" },
    label: { ar: "طبعة 2026", en: "2026 edition" },
    priceMinor: 18500, currency: "EGP",
    availability: "in_stock", stockTotal: 10, stockReserved: 2,
    preOrderEligible: false, sku: "AAR3-2026", updatedAt: ISODate("2026-09-22T10:00:00Z")
  }],
  createdAt: ISODate("2026-09-22T10:00:00Z"),
  updatedAt: ISODate("2026-09-22T10:00:00Z")
}
```

## 32.3 Cart

```
{
  _id: ObjectId("66f00000000000000000000030"),
  ownerType: "guest",
  userId: null,
  sessionId: "hashed-session-scope",
  items: [{
    productId: ObjectId("66f00000000000000000000010"),
    variantId: "edition-2026",
    quantity: 1, unitPriceMinor: 18500,
    productNameSnapshot: { ar: "مرجع اللغة العربية - الصف الثالث" },
    imageSnapshot: "catalog/azhar-arabic-3",
    addedAt: ISODate("2026-09-22T10:05:00Z")
  }],
  currency: "EGP", version: 1,
  expiresAt: ISODate("2026-10-22T10:05:00Z"),
  createdAt: ISODate("2026-09-22T10:05:00Z"),
  updatedAt: ISODate("2026-09-22T10:05:00Z")
}
```

## 32.4 Order

```
{
  _id: ObjectId("66f00000000000000000000040"),
  reference: "AZ-20260922-000041",
  customerId: null,
  guestAccessTokenHash: "sha256:...",
  customerSnapshot: { name: "أحمد علي", email: "ahmed@example.com", phone: "+201000000000" },
  items: [{
    orderItemId: "item-1", productId: ObjectId("66f00000000000000000000010"),
    variantId: "edition-2026", name: { ar: "مرجع اللغة العربية - الصف الثالث" },
    quantity: 1, unitPriceMinor: 18500, lineTotalMinor: 18500,
    selectedAttributes: { edition: "2026" }, stockItemKey: "product:...:variant:edition-2026"
  }],
  totals: {
    productSubtotalMinor: 18500, shippingEstimateMinor: 5000,
    shippingFinalMinor: null, discountMinor: 0, totalMinor: 23500, currency: "EGP"
  },
  fulfillment: {
    method: "delivery",
    addressSnapshot: { governorate: "Qena", city: "Qena", area: "Omar Effendi",
      street: "Example", buildingNumber: "1", floor: "2", apartment: "3",
      landmark: null, recipientName: "أحمد علي", recipientPhone: "+201000000000", notes: null },
    provider: null, shippingStatus: "not_started", estimateSource: "configured",
    finalCostConfirmedAt: null
  },
  paymentMethodKey: "cash_on_delivery", paymentId: null,
  status: "pending_review", statusHistory: [{ status: "pending_review", at: ISODate("2026-09-22T10:10:00Z"), actorId: null }],
  idempotencyKey: "checkout-key-1", version: 1,
  submittedAt: ISODate("2026-09-22T10:10:00Z"),
  createdAt: ISODate("2026-09-22T10:10:00Z"),
  updatedAt: ISODate("2026-09-22T10:10:00Z")
}
```

## 32.5 Payment and proof

```
{
  _id: ObjectId("66f00000000000000000000050"),
  ownerType: "order", ownerId: ObjectId("66f00000000000000000000040"),
  customerId: ObjectId("66f00000000000000000000001"),
  methodKey: "instapay", methodSnapshot: { label: { ar: "إنستاباي" } },
  amountDueMinor: 23500, currency: "EGP",
  status: "under_review", proofRequired: true, proofSubmissionCount: 1,
  createdAt: ISODate("2026-09-22T10:15:00Z"), updatedAt: ISODate("2026-09-22T10:16:00Z")
}
{
  _id: ObjectId("66f00000000000000000000051"),
  paymentId: ObjectId("66f00000000000000000000050"),
  ownerType: "order", ownerId: ObjectId("66f00000000000000000000040"),
  customerId: ObjectId("66f00000000000000000000001"),
  submissionNumber: 1,
  files: [{ cloudinaryPublicId: "private/payment-proofs/2026/09/abc",
    resourceType: "image", format: "jpg", bytes: 123456, width: 1080, height: 1920 }],
  status: "under_review", reviewedBy: null, reviewedAt: null,
  createdAt: ISODate("2026-09-22T10:16:00Z")
}
```

## 32.6 Service request and quotation

```
{
  _id: ObjectId("66f00000000000000000000060"),
  reference: "SR-20260922-000007", customerId: ObjectId("66f00000000000000000000001"),
  customerSnapshot: { name: "أحمد علي", phone: "+201000000000", email: "ahmed@example.com" },
  serviceCategoryId: ObjectId("66f00000000000000000000070"),
  serviceCategorySnapshot: { slug: "printing", name: { ar: "طباعة" }, formVersion: 1 },
  submittedFields: {}, description: "أحتاج طباعة مستندات",
  status: "admin_review", quotationId: null, paymentId: null,
  createdAt: ISODate("2026-09-22T10:20:00Z"), updatedAt: ISODate("2026-09-22T10:20:00Z")
}
{
  _id: ObjectId("66f00000000000000000000061"),
  serviceRequestId: ObjectId("66f00000000000000000000060"),
  customerId: ObjectId("66f00000000000000000000001"),
  version: 1, amountMinor: 7500, currency: "EGP", status: "sent",
  sentBy: ObjectId("66f00000000000000000000002"),
  createdAt: ISODate("2026-09-22T11:00:00Z"), updatedAt: ISODate("2026-09-22T11:00:00Z")
}
```

## 32.7 Pre-order, return, refund, notification, audit

```
{
  reference: "PO-20260922-000003", customerId: ObjectId("66f000000000000000000001"),
  productId: ObjectId("66f0000000000000000000010"), variantId: "edition-2027",
  quantity: 1, capturedPriceMinor: 20000, currency: "EGP",
  status: "admin_review", expectedAvailabilityAt: null, paymentId: null
}
{
  reference: "RET-20260922-000002", orderId: ObjectId("66f000000000000000000040"),
  customerId: ObjectId("66f000000000000000000001"),
  items: [{ orderId: "item-1", quantity: 1, reason: "damaged_item", eligible: true }],
  status: "return_review", refundId: null
}
{
  orderId: ObjectId("66f000000000000000000040"),
  returnRequestId: ObjectId("66f000000000000000000080"),
  amountMinor: 18500, currency: "EGP", methodKey: "instapay",
  status: "initiated", recordedBy: ObjectId("66f000000000000000000002"),
  recordedAt: ISODate("2026-09-22T12:00:00Z")
}
{
  recipientUserId: ObjectId("66f000000000000000000001"),
  type: "order_status_changed", title: { ar: "تحديث حالة الطلب" },
  body: { ar: "تم تحديث حالة طلبك" },
  entityType: "order", entityId: ObjectId("66f000000000000000000040"),
  readAt: null, channels: ["in_app", "socket"],
  deliveryStatus: "pending", dedupeKey: "order:40:status:accepted:2"
}
{
  actorId: ObjectId("66f000000000000000000002"), actorRole: "admin",
  action: "payment_proof_confirmed", entityType: "paymentProof",
  entityId: ObjectId("66f0000000000000000000051"),
  previousState: "under_review", newState: "confirmed",
  requestId: "req-123", createdAt: ISODate("2026-09-22T12:05:00Z")
}
```

## 33. Example Queries

```
// Active books first, then other published products.
db.products.find({ isPublished: true }).sort({ categoryId: 1, updatedAt: -1 }).limit(24)

// Product search using populated fields; exact ranking is not invented.
db.products.find({
  isPublished: true,
  $or: [
    { nameSearch: { $regex: query, $options: "i" } },
    { author: { $regex: query, $options: "i" } },
    { isbn: queryNormalized },
    { publisher: { $regex: query, $options: "i" } }
  ]
})

// Customer's own order history.
db.orders.find({ customerId }).sort({ submittedAt: -1 }).limit(25)

// Admin payment queue.
db.payments.find({ status: { $in: ["under_review", "new_proof_requested"] } })
  .sort({ updatedAt: 1 }).limit(50)

// Outside-Qena order aggregation.
db.orders.aggregate([
  { $match: { "fulfillment.method": "delivery", "fulfillment.addressSnapshot.governorate": { $ne: "Qena" } } },
  { $group: { _id: "$fulfillment.addressSnapshot.governorate", orders: { $sum: 1 } } }
])
```

## 34. Index Definitions

```
db.users.createIndex({ email: 1 }, { unique: true })
db.users.createIndex({ phone: 1 }, { unique: true })
db.addresses.createIndex({ userId: 1, createdAt: -1 })
db.categories.createIndex({ slug: 1 }, { unique: true })
db.categories.createIndex({ isActive: 1, displayOrder: 1 })
db.products.createIndex({ slug: 1 }, { unique: true })
db.products.createIndex({ categoryId: 1, isPublished: 1, availability: 1 })
db.products.createIndex({ priceMinor: 1, isPublished: 1 })
db.products.createIndex({ isbn: 1 }, { unique: true, partialFilterExpression: { isbn: { $type: "string" } } })
db.products.createIndex({ searchText: "text" }) // validate Arabic behavior before enabling as primary search
db.carts.createIndex({ userId: 1 }, { unique: true, partialFilterExpression: { ownerType: "user" } })
db.carts.createIndex({ sessionId: 1 }, { unique: true, partialFilterExpression: { ownerType: "guest" } })
db.carts.createIndex({ expiresAt: 1 }, { expireAfterSeconds: 0 })
db.orders.createIndex({ reference: 1 }, { unique: true })
db.orders.createIndex({ customerId: 1, submittedAt: -1 })
db.orders.createIndex({ status: 1, submittedAt: 1 })
db.orders.createIndex({ "fulfillment.addressSnapshot.governorate": 1, submittedAt: -1 })
db.orders.createIndex({ idempotencyKey: 1 }, { unique: true, sparse: true })
db.payments.createIndex({ status: 1, updatedAt: 1 })
db.payments.createIndex({ ownerType: 1, ownerId: 1 }, { unique: true })
db.paymentProofs.createIndex({ paymentId: 1, submissionNumber: 1 }, { unique: true })
db.paymentProofs.createIndex({ status: 1, createdAt: 1 })
db.inventoryReservations.createIndex(
  { orderId: 1, orderItemId: 1 },
  { unique: true, partialFilterExpression: { status: "active" } }
)
db.inventoryReservations.createIndex({ productId: 1, variantId: 1, status: 1 })
db.inventoryTransactions.createIndex({ productId: 1, variantId: 1, createdAt: -1 })
db.serviceCategories.createIndex({ slug: 1 }, { unique: true })
db.serviceRequests.createIndex({ reference: 1 }, { unique: true })
db.serviceRequests.createIndex({ customerId: 1, createdAt: -1 })
db.serviceRequests.createIndex({ status: 1, createdAt: 1 })
db.quotations.createIndex({ serviceRequestId: 1, version: -1 })
db.preOrders.createIndex({ reference: 1 }, { unique: true })
db.preOrders.createIndex({ customerId: 1, createdAt: -1 })
db.preOrders.createIndex({ productId: 1, variantId: 1, status: 1 })
db.returnRequests.createIndex({ reference: 1 }, { unique: true })
db.returnRequests.createIndex({ orderId: 1, customerId: 1 })
db.refunds.createIndex({ orderId: 1, createdAt: -1 })
db.notifications.createIndex({ recipientUserId: 1, createdAt: -1 })
db.notifications.createIndex({ recipientUserId: 1, readAt: 1, createdAt: -1 })
db.notifications.createIndex({ dedupeKey: 1 }, { unique: true })
db.coupons.createIndex({ codeNormalized: 1 }, { unique: true })
db.couponRedemptions.createIndex({ couponId: 1, orderId: 1 }, { unique: true })
db.couponRedemptions.createIndex({ createdAt: -1 })
db.contentModules.createIndex({ active: 1, startsAt: 1, endsAt: 1, displayOrder: 1 })
db.settings.createIndex({ key: 1 }, { unique: true })
db.auditLogs.createIndex({ entityType: 1, entityId: 1, createdAt: -1 })
db.auditLogs.createIndex({ actorId: 1, createdAt: -1 })
db.auditLogs.createIndex({ createdAt: -1 })
db.outboxEvents.createIndex({ status: 1, availableAt: 1 })
db.outboxEvents.createIndex({ dedupeKey: 1 }, { unique: true })
```

## 35. Database Testing Strategy

### Unit and schema tests

- Money arithmetic and rounding.
- Product/variant validation and nullable metadata.
- Search behavior when fields are missing.
- Coupon percentage/fixed calculations.
- Address snapshots and no-map-pin constraint.
- Every enum and transition guard.

### Integration tests

- MongoDB Atlas-compatible replica set test environment.
- Unique indexes and duplicate idempotency keys.
- Transaction rollback at every multi-document operation.
- Ownership filters and proof-access restrictions.
- Stock reservation/release/deduction accounting.
- Outbox and notification dedupe.

### Concurrency tests

- Two simultaneous acceptances for the final unit.
- Duplicate checkout with the same key.
- Concurrent proof submissions.
- Concurrent Admin inventory edits.
- Repeated refund completion.

### Data security tests

- No password/token/proof secret in serialized responses or logs.
- Guest token cannot access another order.
- Customer cannot read another customer's order/proof.
- Admin role cannot bypass configured permission.
- Signed Cloudinary URL is short-lived and authorization-checked.

## 36. Traceability Matrix

| Capability          | Source            | Collections / implementation                                                  |
|---------------------|-------------------|-------------------------------------------------------------------------------|
| Guest checkout      | AUTH-001, CHK-001 | <code>carts</code> , <code>orders</code> , guest token hash                   |
| Books-first catalog | CAT-004-009       | <code>categories</code> , <code>products</code> , <code>contentModules</code> |

| Capability              | Source                     | Collections / implementation                                                              |
|-------------------------|----------------------------|-------------------------------------------------------------------------------------------|
| Product variants        | VAR-001-003                | Embedded <code>products.variants</code> , order snapshots                                 |
| Search/filter           | SEARCH-001-004             | Product normalized fields, indexes                                                        |
| Payment proof           | PAY-001-008                | <code>payments</code> , <code>paymentProofs</code> , Cloudinary private media             |
| Pending Review order    | ORD-002, §8.1              | <code>orders.status</code> , transition service                                           |
| Stock reservation       | INV-002-006                | Product counters, <code>inventoryReservations</code> , <code>inventoryTransactions</code> |
| Shipping estimate/final | SHIP-004-009               | Order fulfillment snapshot, <code>settings</code>                                         |
| Services/quotations     | SRV-001-008, QUOTE-001-005 | <code>serviceCategories</code> , <code>serviceRequests</code> , <code>quotations</code>   |
| Pre-orders              | PRE-001-007                | <code>preOrders</code> , linked order/payment                                             |
| Returns/refunds         | RET-001-004, REF-001-003   | <code>returnRequests</code> , <code>refunds</code>                                        |
| Notifications/realtime  | NOT-001-004, RT-001-003    | <code>notifications</code> , <code>outboxEvents</code>                                    |
| RBAC/audit              | RBAC-001-004, PAY-007      | <code>roles</code> , <code>auditLogs</code>                                               |
| Coupons                 | COUP-001-005               | <code>coupons</code> , <code>couponRedemptions</code> , order snapshot                    |
| Open payment conflict   | OD-20                      | <code>settings.payment.methods</code> , method snapshots                                  |

## 37. Open Decisions

| ID    | Affected data/backend                              | Safe current implementation                        | Must finalize before production  |
|-------|----------------------------------------------------|----------------------------------------------------|----------------------------------|
| OD-01 | <code>settings.contact</code>                      | Nullable/configurable contact values               | Final phone/WhatsApp destination |
| OD-02 | <code>users.emailVerifiedAt</code>                 | Store verification; gate only configured actions   | Exact gating scope               |
| OD-03 | <code>settings.shipping</code>                     | Adapter/config interface; no invented rate table   | Rate source/logic                |
| OD-04 | <code>orders.fulfillment.shippingFinalMinor</code> | Keep estimate and final separate                   | Binding point/reconfirmation     |
| OD-05 | Order copy/config                                  | No promised time                                   | Delivery ranges                  |
| OD-06 | Shipping validator                                 | Configurable serviceability response               | Excluded areas                   |
| OD-07 | COD order status                                   | Remain <code>customer_confirmation_required</code> | Timeout/expiry                   |
| OD-08 | Inventory acceptance                               | Atomic conditional reservation                     | Fairness rule                    |
| OD-09 | Inventory settings                                 | No threshold alert                                 | Numeric thresholds               |

| ID    | Affected data/backend                     | Safe current implementation                                              | Must finalize before production                       |
|-------|-------------------------------------------|--------------------------------------------------------------------------|-------------------------------------------------------|
| OD-10 | <code>preOrders.capturedPriceMinor</code> | Capture price, defer honoring policy                                     | Price honoring                                        |
| OD-11 | Pre-order allocation                      | No allocation sequence                                                   | Allocation order                                      |
| OD-12 | <code>serviceCategories.fields</code>     | Versioned configurable fields                                            | Required fields/documents                             |
| OD-13 | Service category                          | No SLA promise                                                           | Turnaround                                            |
| OD-14 | Service payment                           | <code>codAllowed: null</code>                                            | COD applicability                                     |
| OD-15 | Return policy flags                       | Admin eligibility flag only                                              | Periods/non-returnables                               |
| OD-16 | <code>refunds</code> statuses             | Persist manual status, no SLA                                            | Refund SLA                                            |
| OD-17 | Coupon fields                             | Nullable configurable governance                                         | Stacking/limits/expiry/minimums                       |
| OD-18 | Reports                                   | Actual dimensions only                                                   | KPI targets                                           |
| OD-19 | NFR/retention                             | Secure baseline                                                          | Formal accessibility/performance/availability targets |
| OD-20 | Payment settings                          | Data-driven methods; PRD list has functional precedence pending approval | Final methods and cash-at-library choice              |
| OD-21 | Orders/inventory                          | No silent partial acceptance                                             | Multi-item partial behavior                           |

## 38. Implementation Checklist

- ☐ Provision Atlas replica set and least-privilege application user.
- ☐ Implement migration runner, collection validators, and indexes.
- ☐ Implement money/date conventions and typed state enums.
- ☐ Implement users, roles, addresses, and token security.
- ☐ Implement products with embedded variants and opening inventory ledger.
- ☐ Implement carts and order snapshots with idempotency.
- ☐ Implement payment/proof references and private Cloudinary access.
- ☐ Implement transactional reservation, release, and deduction.
- ☐ Implement service requests, quotations, and payment gate.
- ☐ Implement pre-orders without stock movement.
- ☐ Implement returns/refunds as separate records.
- ☐ Implement persisted notifications and outbox dedupe.
- ☐ Implement configurable settings for all Open Decisions.

- [ ] Implement immutable audit logs.
- [ ] Run schema, transaction, concurrency, security, and restore tests.
- [ ] Resolve OD-01 through OD-21 before hard-coding dependent production behavior.

## Requirements reviewed

---

PRD v1.0 functional requirements, state machines, edge cases, acceptance criteria, NFRs, analytics, and OD-01–OD-21; Master Product Brief v2.0 business model, taxonomy, payment/shipping/service constraints; IA/Sitemap route/domain boundaries; Design System and UX/UI data contracts; supplied technical architecture brief.

## Confirmed implementation requirements

---

MongoDB Atlas, UTC dates, EGP money, guest checkout, product/variant support, books-first catalog, manual payment-proof review, no automated gateway, separate service lifecycle, no service upload, free pickup, Admin-selected shipping provider, stock reservation on acceptance, final deduction on Delivered/Picked Up, persisted notifications, RBAC, auditability, and source-of-truth boundaries are represented.

## Technical recommendations introduced

---

Embedded product variants, order/customer snapshots, `settings`, `couponRedemptions`, `outboxEvents`, conditional atomic inventory updates, opaque guest access tokens, signed Cloudinary retrieval, and Atlas Search as a future scale option. These are implementation mechanisms, not business promises.

## Open Decisions preserved

---

OD-01 through OD-21 remain explicit. No shipping rate, delivery SLA, return period, payment-method conflict, service field, service COD rule, coupon governance, timeout, allocation, partial-acceptance, or KPI target was silently resolved.

## Potential schema/backend gaps

---

Final service form fields, shipping-rate provider, customer binding to an adjusted shipping cost, payment method details, content ownership, retention periods, and exact role permissions still require owner input.

## Potential contradictions

---

The Master Brief's four active payment methods conflict with the PRD's six-method functional list. The model supports both through configuration and snapshots; it does not claim the conflict is resolved.

## Security risks

---

Payment-proof media, guest order access, PII exposure, token storage, Cloudinary URL leakage, IDOR, and audit tampering require integration/security tests before launch.

## Implementation ambiguities

---

Transaction behavior for partial acceptance, refund failure retries, service quotation expiry, customer shipping re-confirmation, and verification gating remain policy-dependent.

## Final consistency status

---

**PASS WITH OPEN DECISIONS.** The schema is aligned with the supplied routes, UI contracts, state machines, inventory rules, payment-proof boundary, service/product separation, audit model, idempotency, and concurrency needs. Production implementation must not finalize the listed policy-dependent behavior until the corresponding Open Decisions are approved.